

Informe Proyecto 1

Predicción de popularidad en canción – Grupo 26

Introducción

El proyecto que se presenta a continuación aborda el problema de predicción de la variable de popularidad de canciones a partir de variables provenientes de Spotify como duración de la canción, acusticidad y tempo, entre otras.

El problema se plantea como una regresión supervisada cuyo objetivo es predecir un valor de popularidad continuo entre 0 y 100. Para lo anterior se evaluarán diferentes modelos basados en árboles de decisión.

El enfoque de la solución consta de las siguientes etapas:

- Preprocesamiento de datos: incluye limpieza y transformación de variables según cada modelo a utilizar.
- Evaluación de modelos basados en árboles.
- Selección del mejor modelo.
- Ajuste y calibración de hiperparámetros.
- Entrenamiento final y evaluación de desempeño en el set de datos de Kaggle.

Análisis exploratorio de los datos

Las variables del dataset se dividen en dos grupos según su naturaleza: variables numéricas (continuas y discretas) y variables categóricas tipo string. A continuación, se presenta la clasificación:

Variable	Tipo	Descripción
duration_ms	Numérica continua	Duración de la canción en milisegundos
danceability	Numérica continua	Qué tan bailable es la canción (0-1)
energy	Numérica continua	Intensidad y actividad percibida (0-1)
loudness	Numérica continua	Volumen promedio en decibeles
speechiness	Numérica continua	Presencia de palabras habladas (0-1)
acousticness	Numérica continua	Probabilidad de que la canción sea acústica (0-1)
instrumentalness	Numérica continua	Probabilidad de que sea instrumental (0-1)
liveness	Numérica continua	Probabilidad de presencia de audiencia en vivo (0-1)
valence	Numérica continua	Positividad musical percibida (0-1)
tempo	Numérica continua	Tempo estimado en BPM
popularity	Numérica continua	Variable objetivo (0-100)
key	Numérica discreta	Tonalidad de la canción
mode	Numérica discreta	Modalidad: 1 mayor, 0 menor
time_signature	Numérica discreta	Compás de la canción
explicit	Booleana	Si contiene contenido explícito
track_id	Categórica string	Identificador único del track

artists	Categorica string	Nombre del artista o artistas
album_name	Categorica string	Nombre del álbum
track_name	Categorica string	Nombre de la canción
track_genre	Categorica string	Género musical

Decisiones sobre variables numéricas:

Como todos los modelos a evaluar están basados en árboles de decisión, se decide no realizar escalamiento de variables. Esto se considera innecesario ya que los splits de los árboles no dependen de la magnitud de las variables ni de la diferencia entre estas.

En particular, explicit, key, mode y time_signature son variables discretas o categóricas numéricas, pero dadas las características de los árboles, en las que no se asumen relaciones lineales ni distancias entre los valores— se decide no realizar ningún tratamiento especial.

Únicamente con fines interpretativos se opta por transformar la variable duration_ms de milisegundos a minutos, lo cual no afecta el rendimiento del modelo pero facilita el análisis posterior.

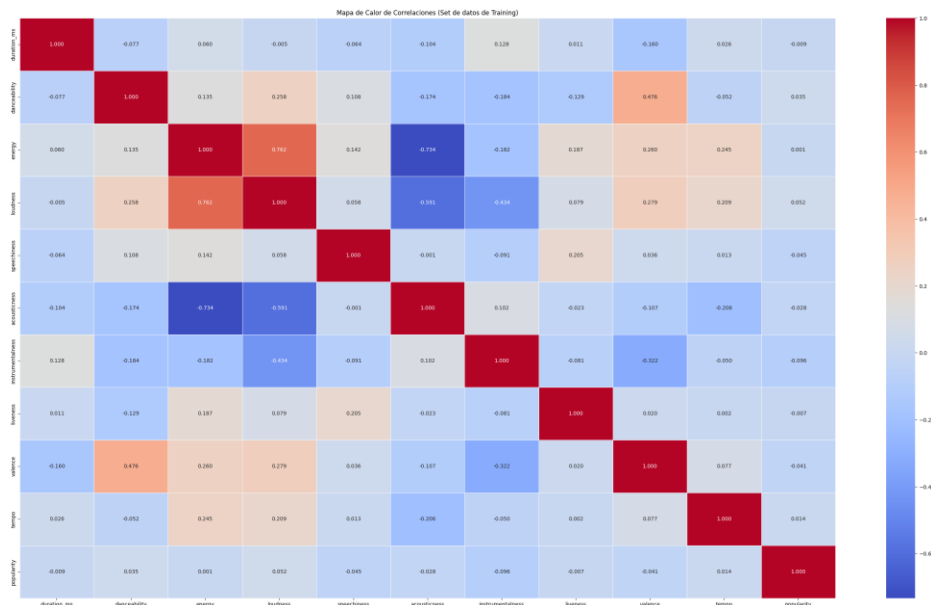
Decisiones sobre variables categóricas tipo string:

Uno de los retos más importantes del proyecto es el manejo de las variables categóricas tipo string, debido a que algunas contienen información relevante para la predicción pero no todos los modelos las soportan de manera nativa. Las variables involucradas son: track_id, artists, album_name, track_name y track_genre.

Para abordar este reto se evaluaron distintas estrategias de tratamiento (eliminación, get_dummies, frequency encoding y target encoding), generando varios sets de datos que se compararán en la sección de selección de modelo.

Análisis de correlación entre variables numéricas:

Se generó un mapa de calor de correlaciones entre las variables numéricas del set de entrenamiento.



Se observa una correlación alta positiva entre energy y loudness (0.76), y una correlación alta negativa entre energy-acousticness (-0.73) y loudness-acousticness (-0.59). Estas relaciones tienen sentido musicalmente: canciones con mayor energía tienden a ser más fuertes y menos acústicas. Los modelos basados en árboles son robustos ante esta multicolinealidad ya que eligen la mejor variable para cada partición. Respecto a popularity, ninguna variable numérica presenta correlación lineal fuerte (todas por debajo de 0.10 en valor absoluto), lo cual sugiere que la señal predictiva está principalmente en las variables categóricas y en interacciones no lineales.

Preprocesamiento de datos

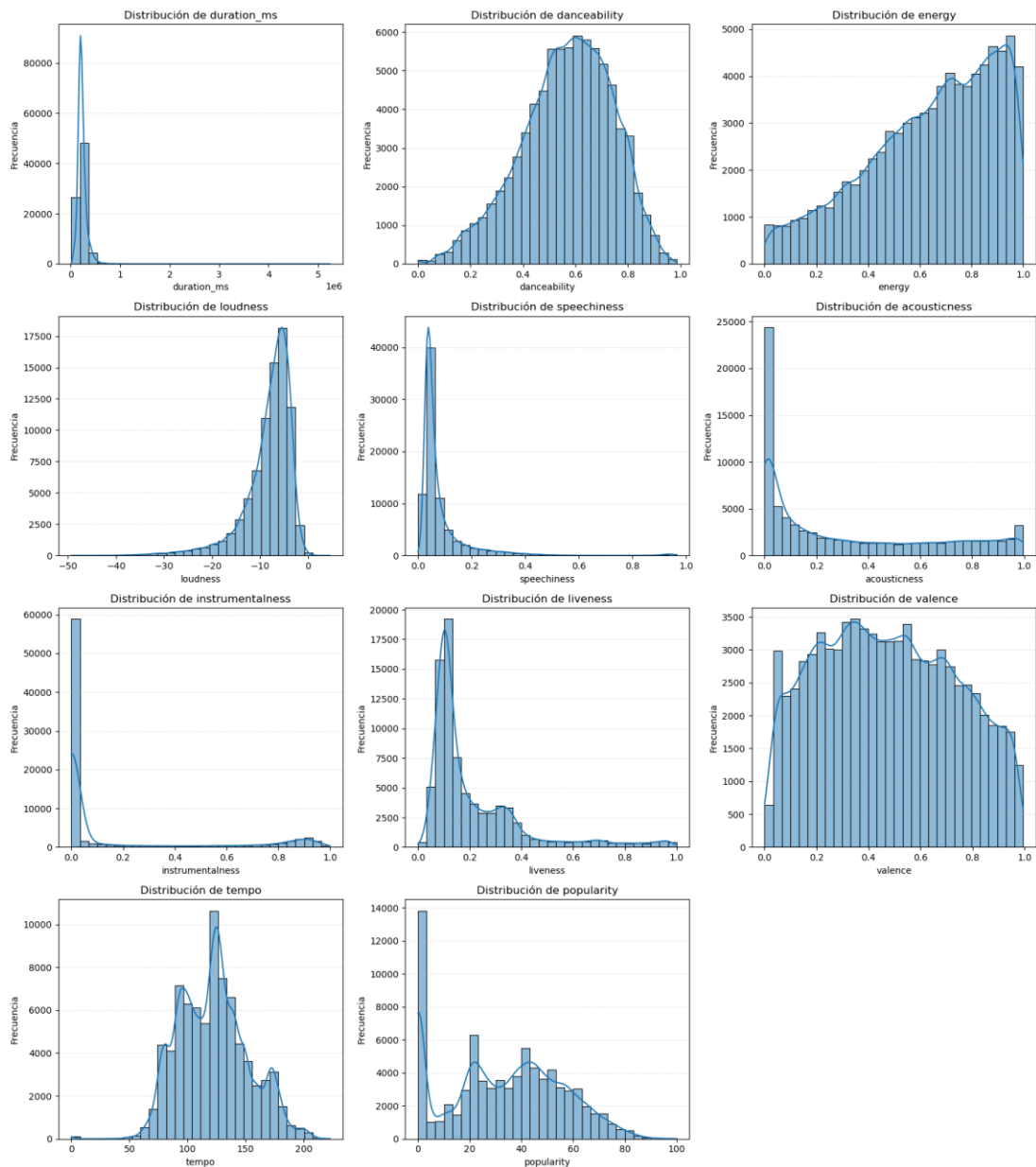
A partir del análisis exploratorio se definieron los pasos de preprocesamiento. Estos buscan limpiar los datos, generar variables informativas y crear distintos sets de entrenamiento que permitan comparar el desempeño de los modelos bajo diferentes estrategias de tratamiento de variables categóricas.

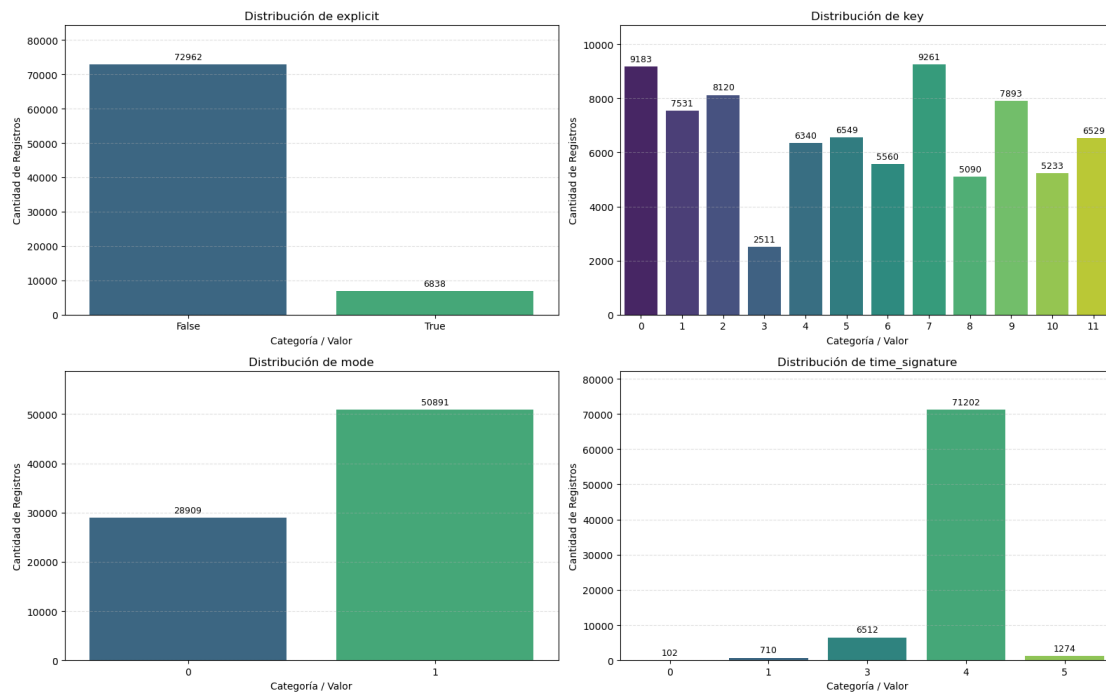
3.1 Limpieza inicial

- Duplicados: se identifican 0 duplicados en Train y 40 en Test, por lo cual se mantiene la información sin modificar.
- Valores nulos: en Test se encuentra una observación con artists, album_name y track_name nulos. Estos valores se reemplazan por 'Unknown' para evitar errores en la ejecución de los modelos.
- Eliminación de columnas no informativas: se eliminan 'Unnamed: 0' (índice irrelevante) y 'track_id' (identificador único sin valor predictivo).

3.2 Hallazgos del análisis

Mediante histogramas, mapas de correlación y estadísticas descriptivas se identifican los siguientes puntos relevantes para el modelado:





- La mayoría de las variables continuas presentan sesgo hacia la izquierda y outliers (especialmente `duration_ms`), lo cual refuerza la elección de modelos basados en árboles, robustos ante valores extremos.
- Existen correlaciones altas entre `energy`, `loudness` y `acousticness` (≈ 0.7 y -0.7). Los modelos basados en árboles manejan esta multicolinealidad sin problema al elegir la mejor variable para cada partición.
- La variable `popularity` está sesgada hacia valores bajos (cuartil 75% = 50), lo cual el modelo deberá manejar.
- `track_genre` presenta buena diversidad (≈ 100 categorías balanceadas), mientras que `artists`, `album_name` y `track_name` tienen alta cardinalidad (miles de valores únicos), lo que descarta one-hot encoding por la maldición de la dimensionalidad.

3.3 Ingeniería de variables

- `duration_min`: conversión de `duration_ms` a minutos para facilitar interpretación.
- `is_collaboration`: variable binaria que indica si la canción tiene varios artistas (detectando el separador ";").
- `explicit`: se transforma de booleana a entero binario para compatibilidad con los modelos.
- `artist_popularity_avg`: variable numérica que asigna a cada canción la popularidad promedio de su artista. Para colaboraciones se promedian los valores de los artistas involucrados; para artistas no vistos en Train, se asigna la media global. El diccionario se construye únicamente con datos de Train para evitar data leakage.

Nota: Posteriormente a estas transformaciones, se divide el set de entrenamiento en Train y Validation con una proporción 80/20, lo cual permite validar el desempeño del modelo antes de generar las predicciones finales sobre Test.

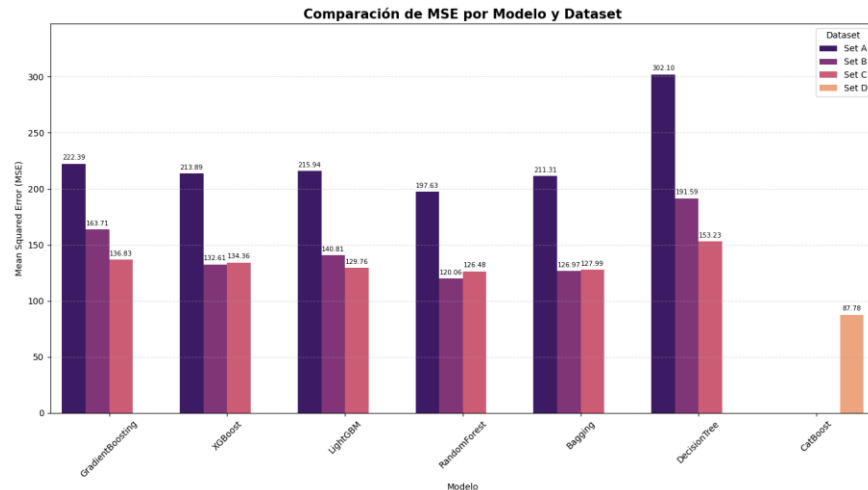
3.4 Generación de sets de datos para comparación de modelos

Con el objetivo de identificar la mejor estrategia de tratamiento de variables categóricas, se generan cuatro sets de datos:

- Set A: se eliminan las variables categóricas y se aplica `get_dummies` sobre `track_genre`.
- Set B: se aplica frequency encoding sobre las variables categóricas, eliminándolas posteriormente.
- Set C: se aplica target encoding sobre las variables categóricas, eliminándolas posteriormente.
- Set D: se mantienen las variables categóricas tal cual, sin transformaciones adicionales. Debido a esta razón solo se utiliza para el modelo de catboost ya que los otros tienen encodings o dejan de usar variables categóricas que es la base para este algoritmo.

3.5 Evaluación preliminar y selección del modelo base

Se evalúan los modelos GradientBoosting, XGBoost, LightGBM, RandomForest, Bagging, DecisionTree y CatBoost sobre los cuatro sets de datos, sin calibración de hiperparámetros, usando MSE como métrica.



La mejor combinación resulta ser CatBoost - Set D, con un MSE de 86.83 en validación, superando ampliamente al resto. Este resultado muestra dos conclusiones importantes:

- Las variables categóricas son fundamentales para la predicción: el rendimiento del Set B (frequency encoding) supera al Set A (sin categóricas), confirmando que esta información no puede descartarse.
- CatBoost maneja de forma nativa las variables categóricas a través de ordered boosting, lo que le permite extraer patrones de artists, album_name, track_name y track_genre sin necesidad de transformaciones manuales, siendo más efectivo que las codificaciones aplicadas sobre los demás modelos.

Con base en estos resultados, se selecciona CatBoost como modelo final por las siguientes razones: en la comparación preliminar de la sección 3.5 obtuvo el menor MSE en validación (87.78) frente a GradientBoosting, XGBoost, LightGBM, RandomForest, Bagging y DecisionTree, con una diferencia significativa respecto al segundo mejor; maneja de forma nativa las variables categóricas mediante el algoritmo de ordered boosting, lo cual evita las pérdidas de información que generaría aplicar one-hot encoding sobre variables de alta cardinalidad como artists o album_name (>20.000 valores); cuenta con mecanismos internos de regularización (l2_leaf_reg, ordered boosting) que lo hacen robusto frente al sobreajuste, y asimismo permite la calibración de bastantes hiperparametros para poder solucionar problemáticas del datalaset desde la misma configuración del modelo.

Calibración

Después de obtener CatBoost como mejor método, se realizó una calibración de los hiperparámetros de este modelo. CatBoost cuenta con una gran cantidad de hiperparámetros, lo que hace inviable calibrarlos todos. Por esta razón, se realizó la calibración de los principales y de aquellos que, según nuestro criterio, tendrían un mayor impacto en la predicción del modelo.

Para la calibración se utilizó un método de calibración individual y secuencial. En este sentido, se ordenaron los hiperparámetros según la importancia que considerábamos tendrían frente a la predicción del modelo. En ese orden, se calibró el primero y, con el resultado obtenido, se fijó su valor para la calibración del segundo. De esta forma, la última calibración tenía n-1 hiperparámetros fijos, siendo n el total de hiperparámetros escogidos y presentados a continuación. Consideramos que este es el mejor método de selección debido a que permite enfocarse en cada uno y entender qué efecto está teniendo sobre el modelo para ir ajustándolo, en lugar de un grid search que, en un dataset tan grande, generaría una carga computacional importante y además llegaría a una combinación cuyo origen sería poco claro de interpretar.

n_estimators: hace referencia a la cantidad de árboles que se evaluarán. Este es uno de los hiperparámetros más importantes ya que permite captar mayores patrones al hacer más iteraciones. Sin embargo, se debe tener cuidado debido a que puede generar que el entrenamiento o la calibración sean muy demorados. Se realizó la calibración individual manteniendo por defecto los demás parámetros. En esta se encontró que entre más aumentaba la cantidad de

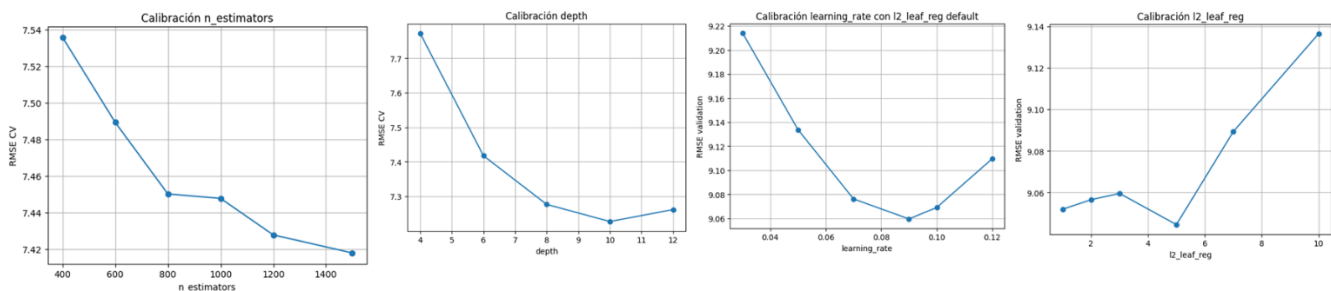
estimadores, más se reducía el RMSE en validación cruzada. Se realizaron iteraciones con los valores (400, 600, 800, 1000, 1200, 1500), encontrando el menor RMSE en 1500 estimadores. En esta variable se decidió no hacer más iteraciones porque, aunque la lógica indica que seguiría disminuyendo, se observa una pendiente cada vez menos pronunciada por lo que el cambio sería mínimo y aumentaría considerablemente la carga computacional.

Depth: controla la profundidad máxima de los árboles escogidos previamente con el número de estimadores. Este parámetro es importante ya que, con la profundidad, se pueden capturar patrones más complejos y obtener un mejor resultado. Se realizó la calibración con los valores (4, 6, 8, 10, 12) y se observó que en 10 se llega al RMSE mínimo. A partir de este punto el RMSE comienza a aumentar nuevamente, presentando un comportamiento similar a una función cuadrática donde el mínimo corresponde al valor escogido.

learning_rate: indica la magnitud del paso en el proceso de optimización. Pasos muy altos generarían un avance mayor pudiendo saltarse el óptimo, mientras que pasos muy pequeños aumentarían la carga computacional. En este se encontró un comportamiento cuadrático, donde el modelo llega a un RMSE mínimo en 0.09 y comienza a aumentar. Por esto, valores muy bajos en las iteraciones ya establecidas no alcanzan a converger al óptimo, mientras que valores muy altos pueden estar omitiéndolo.

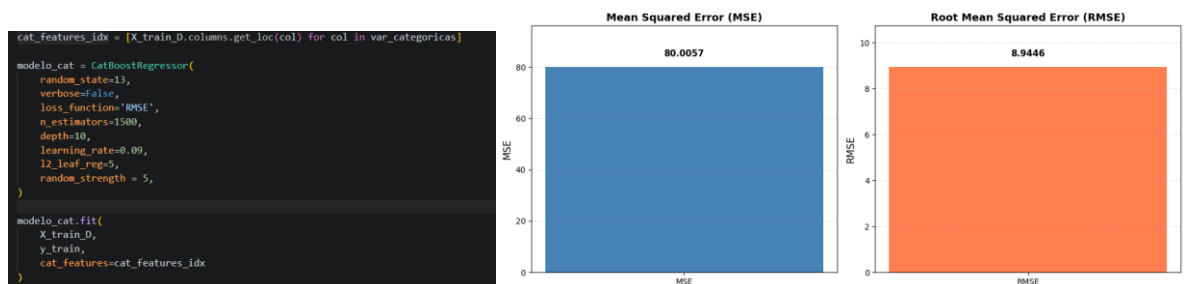
l2_leaf_reg: este hiperparámetro se eligió ya que, al estar trabajando con altas iteraciones, permite generar pesos de penalización a las hojas de cada árbol para prevenir el sobreajuste. Se calibró en este punto debido a que ya se tenían fijados las iteraciones, la profundidad y el paso; entonces, para evitar que esa combinación llegara a un overfitting, se decidió establecer su calibración. En esta se observa que el mejor RMSE se obtiene en 5, indicando una regularización moderada. Este valor representa el mínimo antes de un pico muy alto, lo que indica que regularizaciones más fuertes estarían penalizando de más y reduciendo la capacidad del modelo.

random_state: hace referencia a la semilla generadora de números aleatorios. En este sentido, se tomaron 20 semillas y, con todos los hiperparámetros ya establecidos, se iteró para determinar el conjunto de números que favoreciera el desempeño. En este caso se encontró que 13 era la semilla que lograba el óptimo. Para este hiperparámetro no se presenta gráfica debido a que no son valores con patrones, sino simplemente la semilla que mejor se adaptara.



Entrenamiento del modelo

De acuerdo con los puntos anteriores, se optó por CatBoost como mejor modelo y los hiperparámetros `n_estimators=1500`, `depth=10`, `learning_rate=0.09`, `l2_leaf_reg=5` y `random_state=13`. Con este modelo se puede observar el siguiente comportamiento. Asimismo, en este modelo es muy importante pasarle en el fit las variables categóricas con el fin de que aplique la optimización automática que ofrece CatBoost.



En los resultados se puede observar que el modelo, con el preprocesamiento y los parámetros óptimos, tiene un MSE de 80.0057 y un RMSE de 8.9446 en el set de validación. En este sentido, se puede decir que aproximadamente en cada predicción el modelo se está equivocando en 8.9 unidades de popularidad, sabiendo que la escala es de 0 a 100 esto

equivale a un error del 8.9% en cada predicción. Aunque parezca un error bajo, consideramos que puede mejorarse: ya teniendo el mejor modelo según lo visto en la comparación de modelos y los hiperparámetros óptimos, se podría hacer nuevamente un preprocesamiento más enfocado en este modelo para lograr una mejora.

Segundo Preprocesamiento:

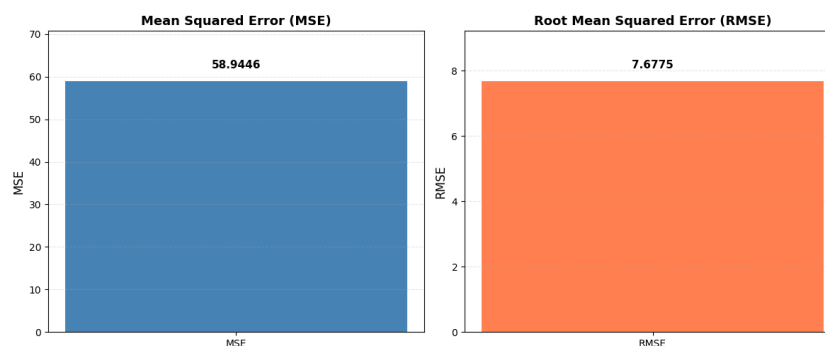
Partiendo de la misma lógica usada en el primer preprocesamiento, se realizó la siguiente metodología:

- Se mantienen las variables creadas (duración de milisegundos a minutos para reducir la escala) aunque esto no afecta al modelo, es más interpretable. Asimismo, se mantiene la variable explicit; ya que se trabaja con CatBoost, no es necesario hacer el ajuste de volverla entera. Tampoco se aplicó ninguna estandarización ni normalización de las variables, debido a que al tratarse de un algoritmo basado en árboles esto no altera los resultados.
- El dataset se partió exactamente igual 80/20 en entrenamiento y validación. Al tener el mismo set para entrenar y comparar, permite observar mejoras en el modelo en el set de validación y, por ende, también al realizar una entrega en Kaggle.
- Se realizó un tratamiento de target encoding con validación cruzada en las variables artista, álbum y género. Esto se realizó debido a que tienen un alto volumen de categorías (25.000, 37.000 y 114 valores únicos respectivamente), lo cual genera que el modelo pueda no capturar patrones y que hacer métodos como one-hot encoding no sea viable porque generaría decenas de miles de columnas. La estrategia de usar validación cruzada con k-fold surge de la idea de evitar leakage, es decir, que las variables no contengan información sobre la variable de respuesta. Al hacer target encoding, se busca tomar una variable categórica y convertirla en una métrica como media, máximo o desviación estándar; sin embargo, si se realiza esto sin hacer particiones, por ejemplo al calcular la media de un artista, la respuesta del propio registro contaría en su propio cálculo. La idea de hacer particiones permite que la media no incluya el registro que se está codificando, evitando así la fuga de información. De esta forma, se realizó la creación de 6 nuevas variables:
 - Artist_pop_mean, Artist_pop_max, Album_pop_mean, Album_pop_std, Genre_pop_mean, Genre_pop_std

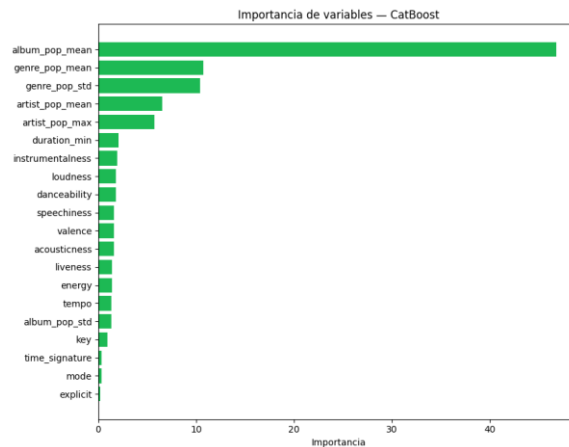
Estas variables permiten observar dónde se centran los datos con los valores de media, por eso están presentes para los 3 grupos (artista, álbum y género). El máximo solo se calcula para artista para identificar cuál es el tope de popularidad de dicho artista, mientras que para álbum y género se usa la desviación estándar; al juntar la media con la dispersión del grupo, se mejora la predicción. Asimismo, en caso de que un artista, álbum o género no se encuentre en el diccionario, permitirá usar la media global de popularidad como imputación.

- Se realizó una normalización de las 3 variables (artista, género y álbum) con el fin de que, a la hora de predecir, no importe si se aplicó un espacio o una mayúscula de más; en ambos casos se hará referencia a lo mismo y, por lo tanto, la predicción será igual.
- Se eliminaron variables redundantes como artists, album_name, track_genre y duration_ms; también se eliminaron variables que deberían ser únicas por registro, como track_id y track_name, debido a que estas no permiten encontrar patrones generalizables.

Posteriormente a este preprocesamiento, se realizó el entrenamiento del modelo siguiendo los mismos parámetros óptimos encontrados en el paso anterior y se obtuvieron los siguientes resultados.



Un MSE de 58.94 y un RMSE de 7.68. En cuanto a los resultados, se puede observar que con este ajuste cada predicción pasa a tener una equivocación del 7.68% en la escala de popularidad, siendo una mejor predicción. Esto equivale a que el modelo mejoró en un 13.7% frente al preprocesamiento elaborado al principio, ya que los hiperparámetros y el modelo son exactamente los mismos. Frente a esto, se puede decir que mejorar la calidad de los datos entregados al modelo puede generar una mejora más profunda, incluso más importante que calibrar los hiperparámetros.



Frente al análisis de importancia se puede observar que las variables originales no tienen un efecto muy grande, lo que sugiere que la falta de creación de nuevas variables en el primer preprocesamiento no permitía obtener una mejora significativa en el modelo a pesar de calibrar varias veces. En esta segunda iteración, la creación de las nuevas variables es la que permitió mejorar el poder predictivo, sobre todo en la variable de álbum, indicando que el álbum al que pertenece una canción es la forma más directa de predecir qué tan popular es. Le siguen la media del género y la desviación de este mismo, lo cual va en línea con el tipo de música que está de moda y siendo más escuchada, generando que solo con el género ya se obtengan algunos puntos por encima que otras canciones. Finalmente, las variables como key, time_signature, mode y explicit resultan ser las menos importantes, casi insignificantes. Se podría evaluar eliminarlas del dataset y realizar nuevamente la predicción; esta no debería verse afectada, sin embargo en el presente caso no se realizará.

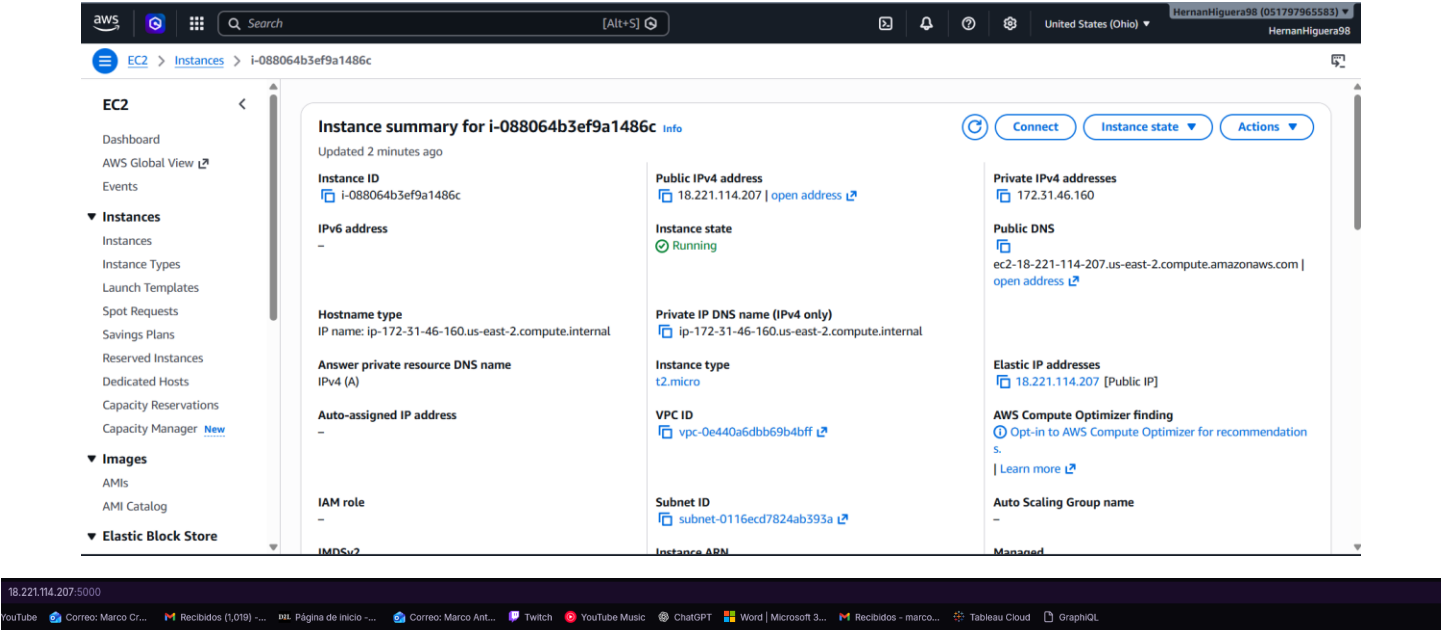
Disponibilización

Para la disponibilización del modelo se desarrolló una API REST utilizando Flask y Flask-RESTX como framework principal. El modelo entrenado, junto con los diccionarios de target encoding (artistas, álbumes y géneros) y las variables del modelo, fueron serializados mediante joblib y cargados al iniciar el servicio, garantizando que cada predicción utilice exactamente el mismo preprocesamiento aplicado durante el entrenamiento.

La API expone un único endpoint GET en la ruta /predict/, el cual recibe como query parameters todas las variables necesarias para realizar una predicción: características de audio de la canción (tempo, loudness, energy, danceability, entre otras), información del artista, álbum y género musical, y la duración en minutos. Internamente, la función crear_df_como_entrenamiento() replica el preprocesamiento del modelo, calculando las variables de target encoding a partir de los diccionarios cargados e imputando con la media global en caso de que el artista, álbum o género no se encuentre registrado.

El servicio fue desplegado en una instancia de AWS, quedando disponible en la dirección <http://18.221.114.207:500/>. Adicionalmente, Flask-RESTX genera automáticamente una interfaz de documentación Swagger accesible desde la raíz de la API, lo que facilita la exploración y prueba de los endpoints sin necesidad de herramientas externas.

En la siguiente imagen se puede observar la instancia EC2 de AWS con estado *running* y la IP pública asignada al servicio. En la siguiente imagen se muestra la interfaz Swagger generada automáticamente por Flask-RESTX, desde donde es posible visualizar el endpoint disponible y los parámetros que acepta. Por último se asigna una IP elástica a la instancia con el fin de contar con una dirección pública fija y así evitar que la IP cambie cada vez que la instancia se detenga o se vuelva a iniciar.



Song Popularity API 1.0

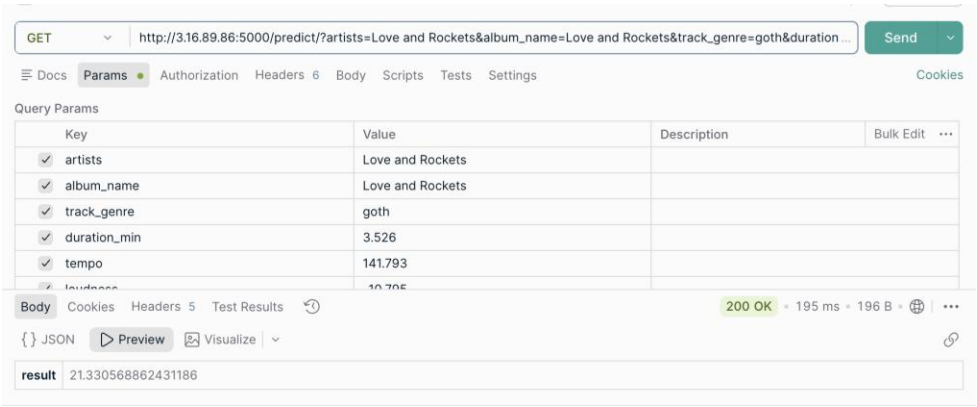
[Base URL: /]
Swagger JSON

Song Popularity Prediction API

predict Song Popularity Regressor

Models

Finalmente, en la siguiente figura se presenta una predicción realizada desde Postman, donde se puede verificar que el servicio responde correctamente retornando el valor de popularidad predicho por el modelo.



Para verificar el correcto funcionamiento de la API desplegada, se realizaron predicciones sobre dos observaciones del set de validación. Los resultados se presentan en la tabla a continuación:

Ca so	artis ts	album_ name	ge nre	dura tion	tem po	lo ud	acou st	instr um	ene rgy	vale nce	spe ech	live	da nce	k ey	mo de	time _sig	expl icit	Predi cción	R ea l
1	Love and Rock ets	Love and Rocket s	got h	3.52 6	141 .79	- 10. 79	0.00 0058	0.05 67	0.8 49	0.32 0	0.05 49	0.4 64	0.3 05	9	1	4	Fals e	21.33	22
2	Filipp a	Filippa Giordan o	op era	3.26 7	83. 69	- 12. 03	0.93 0	0.00 0356	0.1 90	0.13 3	0.03 7	0.0 834	0.2 87	7	0	4	Fals e	20.18	22

Gior dano																			
--------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Link	Result
http://18.221.114.207:5000/predict/?artists=Love%20and%20Rockets&album_name=Love%20and%20Rockets&track_genre=goth&duration_min=3.526&tempo=141.793&loudness=-10.795&acousticness=0.0000582&instrumentalness=0.0567&energy=0.849&valence=0.32&speechiness=0.0549&liveness=0.464&danceability=0.305&key=9&mode=1&time_signature=4&explicit=False	21.33
http://18.221.114.207:5000/predict/?artists=Filippa%20Giordano&album_name=Filippa%20Giordano&track_genre=opera&duration_min=3.267&tempo=83.685&loudness=-12.03&acousticness=0.93&instrumentalness=0.000356&energy=0.19&valence=0.133&speechiness=0.037&liveness=0.0834&danceability=0.287&key=7&mode=0&time_signature=4&explicit=False	22.37

Los resultados evidencian el buen desempeño del modelo en ambos casos. Para Love and Rockets el modelo predijo 21.33 frente a un valor real de 22, un error de apenas 0.67 puntos, mientras que para Filippa Giordano predijo 22.37 frente al mismo valor real, con un error de 0.37 puntos. Esto confirma que el preprocesamiento replicado dentro de la API, incluyendo el target encoding con los diccionarios serializados, funciona correctamente y genera predicciones coherentes con los patrones aprendidos durante el entrenamiento.

Conclusiones

El modelo CatBoost demostró ser la mejor alternativa para la predicción de popularidad de canciones tras comparar su desempeño en una muestra de validación frente a los demás modelos basados en árboles como Random Forest, Gradient Boosting, XGBoost y LightGBM. Su principal ventaja se encuentra en la capacidad nativa de manejar variables categóricas de alta cardinalidad. Adicionalmente, una mejora clave en la capacidad predictiva y de generalización del modelo fue la implementación de target encoding con validación cruzada en las variables de artista, álbum y género, ya que permitió incluir información relevante en el modelo sin incurrir en data leakage.

La calibración secuencial de los hiperparámetros resultó ser una estrategia muy eficiente, ya que permitió comparar y entender el desempeño individual de cada uno de los hiperparámetros y, además, ahorrar poder de cómputo. Para esta sección se determinó que los parámetros óptimos fueron 1500 árboles con profundidad de 8, una tasa de aprendizaje de 0.09, una penalización media de 5 para evitar overfitting y una semilla que mejoraba el rendimiento, siendo la 13.

El segundo preprocesamiento resultó ser un factor determinante para mejorar el desempeño del modelo. Agregar las 6 nuevas variables aplicando target encoding permitió capturar patrones que aumentaron el poder predictivo, lo cual se evidenció en el análisis de importancia, donde las 6 variables creadas lideraban el top, sobre todo la media del álbum, que con diferencia fue la variable más importante en el modelo. Esto logró una mejora de aproximadamente el 13%.

Las pruebas sobre el set de validación demostraron que el modelo en producción se comporta de acuerdo con las métricas de entrenamiento, con errores absolutos muy bajos, inferiores a una unidad de popularidad. De esta forma, se confirma un correcto desarrollo del proyecto.

Finalmente, la disponibilización del modelo mediante una API desplegada en la nube usando los servicios de AWS permitió trasladar la solución a un entorno en el que otros usuarios pueden utilizarlo en tiempo real. Aunque el despliegue en AWS y EC2 es de gran importancia, presenta desafíos relevantes en términos de mantenimiento y gestión de la instancia, como los costos asociados y los posibles cambios en la IP pública, lo cual constituye un reto operativo a la hora de escalar soluciones de Machine Learning, pero que en este caso se corrigió correctamente con una IP elástica.